Prove in campo: avvio e regolazione

Guida pratica per le prove sul robot Demetra. Il documento spiega come avviare tutto il sistema e quale parametro cambiare per ogni problema che si vede. Il funzionamento interno del controllore è spiegato in src/seg_controller_onnx/CONTROLLORE.md, i dettagli su Docker e tmux in guida.md.

I percorsi sono quelli della Jetson: il workspace è /mnt/nvme/emanuele_project/ros2_ws_emanuele. È la copia sulla Jetson di questo repository. Se l'hai clonato con un altro nome, cambia il percorso.

1. Prima di partire

• Codice aggiornato sulla Jetson:

```
cd /mnt/nvme/emanuele_project/ros2_ws_emanuele
git pull
```

• Modello giusto: in src/seg_controller_onnx/seg_controller_onnx/models/ devono esserci lraspp_224x224_30deg.onnx (segmentazione) e rete_bordi.onnx (EdgeNet).

• Build con --symlink-install: il nodo legge i modelli dalla cartella del sorgente e setup.py non li installa, quindi senza symlink potrebbero mancare.

```
source /opt/ros/humble/setup.bash
cd /mnt/nvme/emanuele_project/ros2_ws_emanuele
colcon build --symlink-install
source install/setup.bash
```

• Sicurezza: area libera e stop di emergenza a portata di mano. Il driver ha un watchdog: se non riceve /cmd_vel per 500 ms (cmd_vel_timeout_ms in driver_params.yaml) azzera la velocità. Se il controllore si blocca o viene chiuso, il robot si ferma entro circa mezzo secondo.

2. Avvio: quattro terminali nel container

Avvia il container (docker/run.sh sulla Jetson, vedi guida.md), poi apri ogni terminale con docker exec -it emanuele-ros bash. All'inizio di ogni terminale esegui:

```
source /opt/ros/humble/setup.bash
source /mnt/nvme/emanuele_project/ros2_ws_emanuele/install/setup.bash
```

Terminale A: camera

```
ros2 launch realsense2_camera rs_launch.py camera_name:=camera camera_namespace:=camera
```

Controlla che arrivino le immagini (frequenza diversa da zero):

```
ros2 topic hz /camera/camera/color/image_raw/compressed
```

Terminale B: driver seriale

```
ros2 run demetra_serial_driver driver_node --ros-args \
  --params-file /mnt/nvme/emanuele_project/ros2_ws_emanuele/src/demetra_robot/demetra_serial_driver/config/driver_params.yaml
```

Nel log deve comparire Serial port opened successfully: /dev/ttyUSB0.

Terminale C: controllore

Scegli l'algoritmo con path_finder:

```
ros2 run seg_controller_onnx controller_node --ros-args -p path_finder:=pf5
```

path_finder	Algoritmo nella tesi
pf5	Strip-IRLS (predefinito)
pf_net	EdgeNet
pf_ransac	RANSAC edges
pf_centri	Center points
pf_histogram	Column histogram

All’avvio controlla nel log:

• Path finder: pf5, cioè l’algoritmo scelto.

• Modello ONNX: .../lraspp_224x224_30deg.onnx.

• La riga Provider attivi sessione, che dice se la rete gira su GPU (CUDAExecutionProvider) o su CPU. La riga “Inferenza forzata su CPU” viene scritta sempre e va ignorata.

Terminale D: registrazione

Una bag per ogni prova, con un nome che dica algoritmo e prova:

```
ros2 bag record -o prova_pf5_fila1 \
  /camera/camera/color/image_raw/compressed \
  /cmd_vel \
  /debug/segmentation/compressed \
  /debug/roi/compressed \
  /debug/path/compressed \
  /debug/text \
  /odom
```

Facoltativo: video nel browser

```
ros2 run web_video_server web_video_server --ros-args -p port:=8080 -p address:=0.0.0.0
```

guida.md usa http://<IP_JETSON>:8080/stream?topic=/camera/camera/color/image_raw. Le immagini di debug sono pubblicate solo compresse (/debug/roi/compressed, ...). Per vederle nel browser prova http://<IP_JETSON>:8080/stream?topic=/debug/roi&type=ros_compressed. Non l’ho verificato sul robot: se non funziona, la pagina http://<IP_JETSON>:8080/ elenca i topic disponibili.

Per fermare: Ctrl+C prima nel controllore, poi nella registrazione, nel driver e nella camera.

3. Come cambiare un valore

Ci sono due tipi di valori.

A. Parametri ROS

Si cambiano al lancio, senza ricompilare.

```
ros2 run seg_controller_onnx controller_node --ros-args \
  -p path_finder:=pf5 -p enable_debug_images:=true -p alpha_w:=0.30
```

Sono solo quattro: path_finder, enable_debug_images, alpha_w e alpha_ref_fps.

B. Costanti nel codice

Tutto il resto, cioè velocità, guadagno, filtri e parametri degli algoritmi.

1. Modifica il file: controller_node.py per il controllore; pf5.py, pf_net.py, pf_ransac.py, pf_centri.py o pf_histogram.py per gli algoritmi.

2. Ferma il controllore (Ctrl+C).

3. Con il build --symlink-install basta rilanciarlo. Se hai fatto il build senza symlink, ricompila prima:

```
colcon build --symlink-install --packages-select seg_controller_onnx
source install/setup.bash
```

4. Rilancia il controllore e annota nel quaderno delle prove cosa hai cambiato.

4. Sequenza consigliata delle prove

1. Robot fermo nel filare, ruote sollevate o driver spento.

• Su /debug/segmentation la maschera deve coprire il corridoio.

• Su /debug/roi la ROI deve contenere solo il percorso (sezione 5.1).

• Sposta il robot a mano a destra e a sinistra: con il corridoio a destra dell'immagine raw_error in /debug/text è positivo e w negativo, cioè il robot gira a destra.

2. Prima marcia lenta. In controller_node.py attiva il profilo TEST (v_max = 0.07, w_max = 0.21) e prova un tratto breve e dritto.

3. Velocità di prova. Passa al profilo REALE 2 (v_max = 0.20, w_max = 0.58, quello attivo ora), poi eventualmente a REALE 1 (v_max = 0.30, w_max = 0.87).

4. Regolazione del controllore su Strip-IRLS (sezione 5.3), una modifica alla volta.

5. Confronto tra algoritmi. Stesso filare, stesso punto di partenza, stessi parametri del controllore. Cambia solo path_finder e registra una bag per ognuno. Non cambiare i parametri del controllore tra un algoritmo e l'altro, altrimenti il confronto non vale.

5. Problema → cosa guardare → cosa cambiare

Prima di cambiare un valore, guarda /debug/text e le immagini di debug per capire dove nasce il problema:

• Segmentazione: maschera sbagliata.

• Algoritmo: maschera buona ma retta sbagliata o pf_status = lost.

• Controllore: retta buona ma comando sbagliato.

5.1 ROI e algoritmo (file pf5.py, funzione path_finder)

Problema	Cosa vedi	Cosa cambiare
Nella ROI entrano cielo, struttura della serra o la fine del filare	In /debug/roi la ROI parte troppo in alto e compaiono intervalli falsi in cima	Alzare min_row_ratio (per esempio da 0.30 a 0.36). Cambia a gradini: con 16 strip, tra 0.29 e 0.35 la ROI resta dalla riga 64, da 0.36 a 0.40 parte dalla riga 80
La ROI è troppo bassa e il corridoio visto è corto	Poche strip con il corridoio, pf_status spesso lost	Abbassare min_row_ratio (per esempio a 0.25: ROI dalla riga 48)
Buchi o ombre larghe spezzano il corridoio in basso	Due intervalli affiancati nelle strip basse, retta su metà corridoio	Alzare max_interval_gap (per esempio da 40 a 50)
Aggancia un corridoio vicino o macchie laterali	La retta segue un corridoio accanto o un intervallo troppo largo	Abbassare max_interval_gap (per esempio a 30)
Troppi lost con la maschera buona	pf_status = lost con conf di poco sotto 0.38	Abbassare degraded_conf_threshold (per esempio a 0.30). Non è calibrato
Troppi degraded	conf tra 0.38 e 0.60	Abbassare ok_conf_threshold (cambia solo il filtro, vedi 5.3)
Corridoio molto stretto o lontano non visto	Intervalli scartati	Abbassare min_interval_width (8 px) o min_component_area (500 px)

Non toccare n_strips (16): cambia anche l’altezza della ROI e la finestra dei seed (6 strip).

Le baseline (pf_ransac.py, pf_centri.py, pf_histogram.py) hanno anch’esse min_row_ratio = 0.30, ma senza l’estensione alle strip (ROI dalla riga 67). Se cambi la ROI di Strip-IRLS per il campo, cambiala allo stesso modo nelle baseline che confronti.

EdgeNet (pf_net.py) non ha ROI in ingresso e non ha parametri geometrici. Si possono cambiare solo le soglie di stato: valid_threshold 0.5, min_width 8 px, degraded_conf_threshold 0.38 e ok_conf_threshold 0.60.

5.2 Segmentazione

Problema	Cosa vedi	Cosa fare
La maschera non copre il corridoio o copre le piante	/debug/segmentation sbagliata	Non è un problema dei parametri: annota il caso e registra la bag. Si può solo cambiare il modello (controller_node.py, riga 191)
La rete gira su CPU ed è lenta	Log Provider attivi sessione senza CUDA, inference alto in /debug/text	Verifica onnxruntime-gpu nel container

5.3 Controllore (file controller_node.py)

Problema	Cosa vedi in /debug/text	Cosa cambiare
Il robot oscilla attorno al centro del filare	w cambia segno spesso, filtered_error oscilla	Abbassare kp_lat (da 1.2 a 1.0, poi 0.8)
Rientra troppo lentamente o esce dal filare in curva	filtered_error resta grande a lungo, w piccolo	Alzare kp_lat a passi di 0.1–0.2. Controlla che w_max non limiti (sezione 5.4)
Piccoli scatti di sterzo vicino al centro	filtered_error tra −5 e +5 px con w che si muove	Alzare deadband_px (da 4 a 5–6)
Sterzo a scatti o vibrazioni con retta stabile	raw_error stabile, w a scatti	Abbassare alpha_w (parametro ROS, per esempio 0.20)
Sterzo in ritardo rispetto alla retta	filtered_error segue raw_error con ritardo	Alzare ALPHA_ERR_OK (da 0.20 a 0.25–0.30) o alpha_w
Errore filtrato rumoroso	filtered_error segue tutti i salti di raw_error	Abbassare ALPHA_ERR_OK e ALPHA_ERR_DEGRADED
Si ferma per buchi brevi della maschera	filter = lost_by_pf_status dopo 0.5 s senza errore valido	Alzare hold_timeout (da 0.5 a 0.7–1.0 s)
Continua troppo a lungo dopo aver perso la linea	Va avanti con filter = hold_lost	Abbassare hold_timeout
Ignora una correzione vera e grande	raw_error salta di più di 40 px e il filtro lo segue solo dopo 2 frame	È voluto. Se serve, alzare la soglia di salto jump_thr (40 px con ok, 38 px con degraded, dentro filter_error)

v è sempre v_max: il robot non rallenta in curva né con stato degraded. v_min non è usata.

5.4 Velocità e sterzo massimo (v_max, w_max)

Lo sterzo delle ruote è φ = atan2(0.4·w, 2·v) con limite fisico di 30°. A parità di w, a velocità più alta il robot sterza meno.

v_max [m/s]	w_max per arrivare a 30° [rad/s]
0.07	0.21
0.20	0.58
0.30	0.87
0.50	1.44

• Se cambi v_max, cambia anche w_max con questa tabella. Con un w_max più alto non si ottiene più sterzo, perché il driver limita a 30°.

• Con kp_lat = 1.2 e confidence 1, w arriva a w_max con circa 54 px di errore.

• I valori della tabella sono calcolati con la formula del driver, non misurati sul robot.

5.5 Prestazioni

Problema	Cosa vedi	Cosa cambiare
Frequenza bassa	dt in /debug/text molto sopra 33 ms	Avvia con -p enable_debug_images:=false (non avrai le immagini di debug nella bag)
path_finding alto con pf5	Molti ms per frame	Annotalo: i tempi della versione con 10 iterazioni non sono ancora misurati

Gli alpha dei filtri sono corretti con il dt reale, quindi la costante di tempo in secondi resta uguale anche a frequenze diverse.

6. Cosa annotare per ogni prova

• Data, coltura, filare, ora e condizioni di luce.

• path_finder e i valori diversi dai default (ROI, gap, kp_lat, v_max, w_max, alpha).

• Nome della bag.

• Esito: percorso completato o no, uscite dal filare, interventi manuali, punto in cui è successo.

7. Valori di guida.md da non usare

guida.md contiene valori di una versione precedente del controllore:

• Dice kp_lat = 1.0 e v_max = 0.3, mentre il codice attuale ha kp_lat = 1.2 e il profilo REALE 2 con v_max = 0.20.

• Fa riferimento a ./scripts/run.sh e dice che il container viene rimosso con --rm all’uscita. Nel repository lo script è docker/run.sh, che avvia il container in background con -d, senza --rm, e rimuove i container fermi al nuovo avvio. Sulla Jetson potrebbe esistere uno scripts/run.sh diverso, che da qui non ho potuto vedere: controlla quale usi.

Fa fede questo documento con il codice. Per le procedure di Docker e tmux guida.md resta valida.